

Counter-T



Counter-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

```
1
2 public class CounterT {
3     public static void main(String[] args) throws InterruptedException {
4         Counter counter = new Counter();
5
6         Runnable task = () -> {
7             for (int i = 0; i < 1000; i++) {
8                 counter.increment();
9             }
10        };
11
12        Thread t1 = new Thread(task);
13        Thread t2 = new Thread(task);
14        Thread t3 = new Thread(task);
15
16        t1.start();
17        t2.start();
18        t3.start();
19
20        t1.join();
21        t2.join();
```

OUTPUT

```
Final Counter Value: 3000
```

M-Thread



M-Thread

Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

```
1
2 public class MThreadPrimeCalculator {
3
4     static class PrimeCalculatorTask implements Runnable {
5         private int limit;
6         private String threadName;
7         private long executionTime;
8
9         public PrimeCalculatorTask(String threadName, int limit) {
10             this.threadName = threadName;
11             this.limit = limit;
12         }
13
14         public long getExecutionTime() {
15             return executionTime;
16         }
17
18         public String getThreadName() {
19             return threadName;
20         }
21     }
```

OUTPUT

QUESTIONS 8 / 8 completed

1-10 T



1-10 T

Write a Java program to create a simple thread that prints the numbers from 1 to 10.

PROGRAM EDITOR

Java



Run

Save

```
1
2 public class Main {
3     public static void main(String[] args) {
4         NumberThread thread = new NumberThread();
5         thread.start();
6     }
7 }
8
9 class NumberThread extends Thread {
10     @Override
11     public void run() {
12         for (int i = 1; i <= 10; i++) {
13             System.out.println(i);
14             try {
15                 Thread.sleep(100);
16             } catch (InterruptedException e) {
17                 System.out.println(e);
18             }
19         }
20     }
21 }
```

QUESTIONS 8 / 8 completed

Pro-Consume

PRO-CONSUME

Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR

Java

Run

Save

```
1
2 public class ProConsume {
3
4     public static void main(String[] args) {
5
6         java.util.concurrent.BlockingQueue<Integer> queue = new java.util.concurrent.A
7
8         Thread producer = new Thread(new Producer(queue));
9
10        Thread consumer = new Thread(new Consumer(queue));
11
12        producer.start();
13        consumer.start();
14
15        try {
16            producer.join();
17            consumer.join();
18        } catch (InterruptedException e) {
19            e.printStackTrace();
20        }
21    }
22 }
```

QUESTIONS 8 / 8 completed

Deadlock



DEADLOCK

Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

Java

```
1
2 public class DeadlockDemo {
3
4     public static void main(String[] args) {
5
6         final Object resource1 = new Object();
7         final Object resource2 = new Object();
8
9         Thread thread1 = new Thread(() -> {
10             synchronized (resource1) {
11                 System.out.println("Thread 1: locked resource1");
12                 try { Thread.sleep(100); } catch (InterruptedException e) {}
13                 System.out.println("Thread 1: waiting for resource2");
14                 synchronized (resource2) {
15                     System.out.println("Thread 1: locked resource2");
16                 }
17             }
18         });
19     }
```

QUESTIONS 8 / 8 completed

Wait-Notify

WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR

Java

Run

Save

```
1
2 public class Main {
3     public static void main(String[] args) {
4         SharedBuffer buffer = new SharedBuffer();
5
6         Thread producerThread = new Thread(() -> {
7             try {
8                 for (int i = 1; i <= 10; i++) {
9                     buffer.produce(i);
10                    Thread.sleep(500);
11                }
12            } catch (InterruptedException e) {
13                Thread.currentThread().interrupt();
14            }
15        });
16
17
18        Thread consumerThread = new Thread(() -> {
19            try {
20                for (int i = 1; i <= 10; i++) {
21                    buffer.consume(i);
22                }
23            } catch (InterruptedException e) {
24                Thread.currentThread().interrupt();
25            }
26        });
27
28        producerThread.start();
29        consumerThread.start();
30    }
31 }
```

QUESTIONS 8 / 8 completed

Pool

POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

PROGRAM EDITOR

Java

Run

Save

```
1
2 import java.util.concurrent.ExecutorService;
3 import java.util.concurrent.Executors;
4
5 public class ThreadPoolExample {
6
7     public static void main(String[] args) {
8
9         ExecutorService pool = Executors.newFixedThreadPool(4);
10
11         for (int i = 1; i <= 10; i++) {
12             final int taskId = i;
13             pool.execute(() -> {
14                 System.out.println("Task " + taskId + " is running on thread: "
15                     + Thread.currentThread().getName());
16
17                 try {
18                     Thread.sleep(300);
19                 } catch (InterruptedException e) {
20                     Thread.currentThread().interrupt();
21                 }
22             });
23         }
24     }
25 }
```

QUESTIONS 8 / 8 completed

Join



JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

PROGRAM EDITOR

```
1
2 public class JoinDemo {
3     public static void main(String[] args) {
4         MyThread t = new MyThread();
5
6         System.out.println("Main thread started");
7         t.start();
8
9         try {
10            System.out.println("Main thread waiting for sub-thread to complete");
11            t.join();
12        } catch (InterruptedException e) {
13            System.out.println("Main thread interrupted");
14        }
15
16        System.out.println("Main thread completed");
17    }
18 }
19
20 class MyThread extends Thread {
21
```

QUESTIONS 8 / 8 completed

Join



JOIN

Write a Java program to demonstrate how to use the "join" method to wait for a thread to complete its execution before proceeding with the main thread.

PROGRAM EDITOR

```
1
2 public class JoinDemo {
3     public static void main(String[] args) {
4         MyThread t = new MyThread();
5
6         System.out.println("Main thread started");
7         t.start();
8
9         try {
10            System.out.println("Main thread waiting for sub-thread to complete");
11            t.join();
12        } catch (InterruptedException e) {
13            System.out.println("Main thread interrupted");
14        }
15
16        System.out.println("Main thread completed");
17    }
18 }
19
20 class MyThread extends Thread {
21
```